

MODÜL 09

Transformer ve Attention

Attention Is All You Need

Scaled Dot-Product · Multi-Head · Positional Encoding · Encoder · BERT · GPT

Seviye	Orta-İleri — Matematiksel Derinlik
Framework	NumPy (from scratch) · TensorFlow/Keras · PyTorch
Önceki Modül	Modül 08 — RNN/LSTM/GRU
Sonraki Modül	Modül 10 — GAN ve VAE

Öğrenme Hedefleri

- ✓ Scaled dot-product attention formülünü türetmek ve sıfırdan uygulamak
- ✓ Multi-head attention'ın neden birden fazla kafa kullandığını açıklamak
- ✓ Positional encoding'i sinüzoidal formülle hesaplamak ve görselleştirmek
- ✓ Transformer encoder bloğunu (MHA + FFN + Add&Norm;) uygulamak
- ✓ Causal (decoder) mask'ın nasıl çalıştığını açıklamak
- ✓ BERT ve GPT'nin pre-training hedeflerini karşılaştırmak
- ✓ PyTorch'ta mini transformer kodlamak

Scaled Dot-Product Attention

Formül

$$\text{Attention}(Q,K,V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

Q (Query)

Ne arıyorum? — her token için sorgu vektörü

K (Key)

Neyle eşleşirim? — her token için anahtar vektörü

V (Value)

Eşleşince ne döneceğim? — değer vektörü

$\sqrt{d_k}$ ölçekleme

Büyük d_k 'da dot product büyür $\rightarrow$ softmax doyar $\rightarrow \sqrt{d_k}$ ile böl

Causal mask

Decoder'da: üst üçgen = $-\infty$ $\rightarrow$ gelecek tokenlere attention yok

Multi-Head Attention

Formül

$$\text{MHA} = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O$$

Her kafa

$$\text{head}_i = \text{Attention}(Q \cdot W_i^R, K \cdot W_i^K, V \cdot W_i^V)$$

Boyut

$$d_{\text{model}} = h \times d_k \text{ — toplam boyut değişmez}$$

Neden çok kafa?

Her kafa farklı ilişkiyi öğrenir: sözdizimi, anlam, uzun/kısa bağımlılık

Positional Encoding

Sin formülü

$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_model)})$

Cos formülü

$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_model)})$

Neden?

Transformer sıra sırasız işler — PE konumu ekler

Genelleme

Eğitimde görülmemiş uzunluklara genelleşebilir

BERT vs GPT

Mimari

BERT: Encoder-only | GPT: Decoder-only

Yön

BERT: Çift yönlü | GPT: Tek yönlü (causal)

Pre-training

BERT: Masked LM + NSP | GPT: Causal LM

Hedef

BERT: Anlama (NLU) | GPT: Üretme (NLG)

Fine-tuning

BERT: Sınıflandırma, NER, QA | GPT: Tamamlama, özetleme

Transformer Encoder Bloğu – Tam Akış

Giriş: x (seq_len, d_model)

Adım 1: $MHA = \text{MultiHead-SelfAttention}(x, x, x) - x$ hem Q, hem K, hem V

Adım 2: $x = \text{LayerNorm}(x + MHA) - \text{Residual} + \text{Normalize}$

Adım 3: $FFN_out = FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$

Adım 4: $x = \text{LayerNorm}(x + FFN_out) - \text{Residual} + \text{Normalize}$

Çıkış: x (seq_len, d_model) – boyut değişmedi!

Transformer vs RNN Karşılaştırması

Özellik	RNN / LSTM	Transformer
Uzun bağımlılık	Zayıf (vanishing)	Güçlü (attention direkt)
Paralel hesap	Sıralı (yavaş)	Tam paralel (hızlı)
Seq uzunluğu	Lineer $O(T)$	Karesel $O(T^2)$ bellek
Konum bilgisi	Örtük (sıralı)	Explicit PE gerekli
Eğitim hızı	Yavaş (sıralı)	Çok hızlı (GPU paralel)
Yorumlanabilirlik	Düşük	Attention haritaları
SOTA görevi	Zaman serisi (hâlâ)	NLP, Vision, Audio, Kod

Yaygın Hatalar

Hata 1: Positional encoding eklemeyi unutmak	Transformer sıra bilgisi içermez. PE eklenmezse tüm tokenler konumsuz — model anlamsız sonuçlar verir.
Hata 2: Attention maskesini causal decoder'da kullanmamak	Decoder sırasında gelecek tokenlere bakılmamalı. Causal mask (üst üçgen -inf) olmadan model 'hile' yapar.
Hata 3: d_model'in n_heads'e bölünmediğinden hata almak	$d_k = d_{\text{model}} / n_{\text{heads}}$ tam sayı olmalı. $d_{\text{model}}=64$, $n_{\text{heads}}=6 \rightarrow$ HATA. 8 kullan.
Hata 4: LayerNorm'u Add'den önce veya sonra yapmak	Orijinal: $x = \text{LN}(x + \text{sublayer}(x))$ — Post-norm. Bazı uygulamalar Pre-norm: $x = x + \text{sublayer}(\text{LN}(x))$. Pre-norm genellikle daha kararlı.
Hata 5: Attention ağırlıklarını $\sqrt{d_k}$ ile ölçeklememek	Büyük d_k 'da $Q@K.T$ büyük değerler üretir $\rightarrow$ softmax doyar $\rightarrow$ gradyan yok olur. Her zaman $/ \sqrt{d_k}$ böl.

Kaynaklar

- [1] Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS. arXiv:1706.03762.
- [2] Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers. NAACL.
- [3] Radford, A. et al. (2018). Improving Language Understanding by Generative Pre-Training (GPT). OpenAI.
- [4] Ba, J.L. et al. (2016). Layer Normalization. arXiv:1607.06450.
- [5] Alammar, J. (2018). The Illustrated Transformer.
<https://jalammar.github.io/illustrated-transformer/>
- [6] CS224n Stanford — NLP with Deep Learning. <http://web.stanford.edu/class/cs224n/>
- [7] Andrej Karpathy — minGPT. <https://github.com/karpathy/minGPT>

Sonraki Adım: Modül 10 — GAN ve VAE